PROJECT REPORT

SAFE-Drive

A Research Toolkit for AI-Assisted Road Safety Monitoring

Keywords: Road Safety · Computer Vision · Intelligent Transportation Systems · Vehicle Tracking · Speed Estimation · Near-Miss Detection · Flask · Risk Assessment

Repository: github.com/bhuvzzs24-spec/SAFE-Drive

Contents

Abstract....3
1. Introduction....3
2. Problem Statement....3
3. Objectives....4
4. Existing System....4
5. Proposed System....4
6. System Architecture....5
7. Tools and Technologies....8
8. AI Models and Algorithms....8
9. Backend Implementation....10
10. Testing and Validation....11
11. Results and Discussion....12
12. Limitations....13
13. Future Scope....13
14. Conclusion....13
References....14

Abstract

Road safety monitoring is an important application of computer vision and intelligent transportation systems. Reviewing long traffic recordings by hand is slow, which makes it hard to find potentially dangerous interactions. SAFE-Drive is a research toolkit that analyses traffic videos, tracks road users, estimates vehicle speed under appropriate calibration, and screens trajectories for potential near-miss situations using closest-approach and time-to-collision indicators.

The system combines a Flask review dashboard with computer vision modules, road-plane calibration, configurable risk scoring, and persistent event storage. It accepts offline video uploads and live RTSP/RTSPS camera streams, and a human-review workflow lets recorded events be marked as verified or rejected. SAFE-Drive is a research prototype, not an enforcement system: its detection, tracking, speed and near-miss performance still need evaluation on representative footage with independently verified ground truth.

1. Introduction

Road traffic safety remains a significant challenge in urban environments, where vehicles, pedestrians, and other road users interact under varying conditions. Conventional monitoring depends on human observation and manual footage review, which gives useful context but is time-consuming for extended recordings. Computer vision can extract information about road users and their movement directly from traffic video.

SAFE-Drive organizes this analysis in one interface: detection and tracking, calibrated trajectory analysis, speed estimation, closest-approach and time-to-collision screening, configurable risk scoring, and event review. Its main purpose is to help researchers find road-user interactions that may need further safety review, rather than relying only on manual inspection. Risk scores are treated as indicators for investigation, never as evidence of an accident, a legal violation, or individual responsibility.

1.1 Potential Applications

- Traffic-video analysis and road-user trajectory studies
- Speed-estimation research
- Near-miss screening and traffic safety research
- Intelligent transportation systems and human-reviewed video analytics

2. Problem Statement

Finding potentially dangerous interactions in long traffic recordings takes substantial manual effort. Speed estimates taken directly from image coordinates are also unreliable when camera perspective and real-world road geometry are ignored.

A research system is therefore needed that organizes traffic-video analysis, maintains road-user trajectories, estimates speed when valid calibration exists, flags potentially hazardous interactions, and provides a structured interface for reviewing the results. SAFE-Drive addresses this by combining modular vision components with geometric analysis, configurable risk indicators, and persistent event records.

3. Objectives

1. To develop a dashboard for analysing uploaded traffic videos and supported live IP camera streams.
2. To integrate vehicle and pedestrian detection with multi-object tracking.
3. To estimate vehicle speed from timestamped trajectories and camera-specific road-plane calibration.
4. To screen trajectories for potential near-miss situations using closest-approach and time-to-collision indicators.
5. To generate configurable, explainable risk scores from available signals.
6. To store sessions, tracks, events, and review decisions for later inspection.
7. To support human review of events instead of automated enforcement.
8. To lay a foundation for future evaluation with annotated datasets and independent ground truth.

4. Existing System

Conventional traffic monitoring relies on manual CCTV observation, separate speed-measurement systems, and independent review processes, so operators must switch between tools and locate events by hand. Basic computer vision implementations may estimate speed from fixed pixel-to-distance ratios or flag interactions from incomplete trajectory data, which can mislead when perspective, calibration quality, timing, and uncertainty are ignored.

SAFE-Drive instead combines geometric calibration, timestamp-based trajectory analysis, risk screening, event persistence, and human review in one modular application. It is a research prototype and has not been shown to be more accurate than existing traffic-monitoring systems.

5. Proposed System

SAFE-Drive provides a Flask dashboard connected to modular analysis components. A user uploads a traffic video or configures a supported live camera stream; when the vision dependencies and compatible model weights are available, the pipeline detects road users and associates detections across frames. The resulting trajectories feed calibrated speed estimation and pairwise interaction analysis, and a configurable risk engine turns the indicators into reviewable events.

Table 1. *Major modules of SAFE-Drive*

Module	Function
5.1 Video input	Accepts uploaded footage and RTSP/RTSPS live streams; live analysis needs a reachable, compatible stream.
5.2 Detection and tracking	YOLO-based detection of supported road users and DeepSORT tracking with session-local track IDs.
5.3 Road-plane calibration	A homography relates image coordinates to measured road coordinates; at least four well-distributed, non-collinear point pairs are required, and calibration is specific to one camera view.
5.4 Speed estimation	Uses timestamped, calibrated positions; warm-up and calibration-quality states flag unavailable or unsuitable estimates. Speed rules use a limit, tolerance, and minimum exceedance duration.
5.5 Near-miss screening	Analyses pairs of trajectories with closest-approach and time-to-collision indicators to flag interactions for review.
5.6 Risk assessment	Combines available signals using configurable scoring rules and reports missing signals.
5.7 Event management and review	Stores sessions, tracks, and events; reviewers mark events verified or rejected. Decisions never issue fines automatically.
5.8 Web dashboard	Flask interface for choosing sources, starting analysis, viewing annotated output, and reviewing events; for local use only in its current form.

6. System Architecture

SAFE-Drive follows a layered, modular architecture. Video enters through the input layer, passes through perception (detection and tracking) and analysis (calibration, speed, screening, risk), and is then persisted and presented to a human reviewer. Separating these layers supports independent testing and future improvement. Several stages depend on optional dependencies, trained model weights, or valid camera-specific calibration.

6.1 Layered Architecture

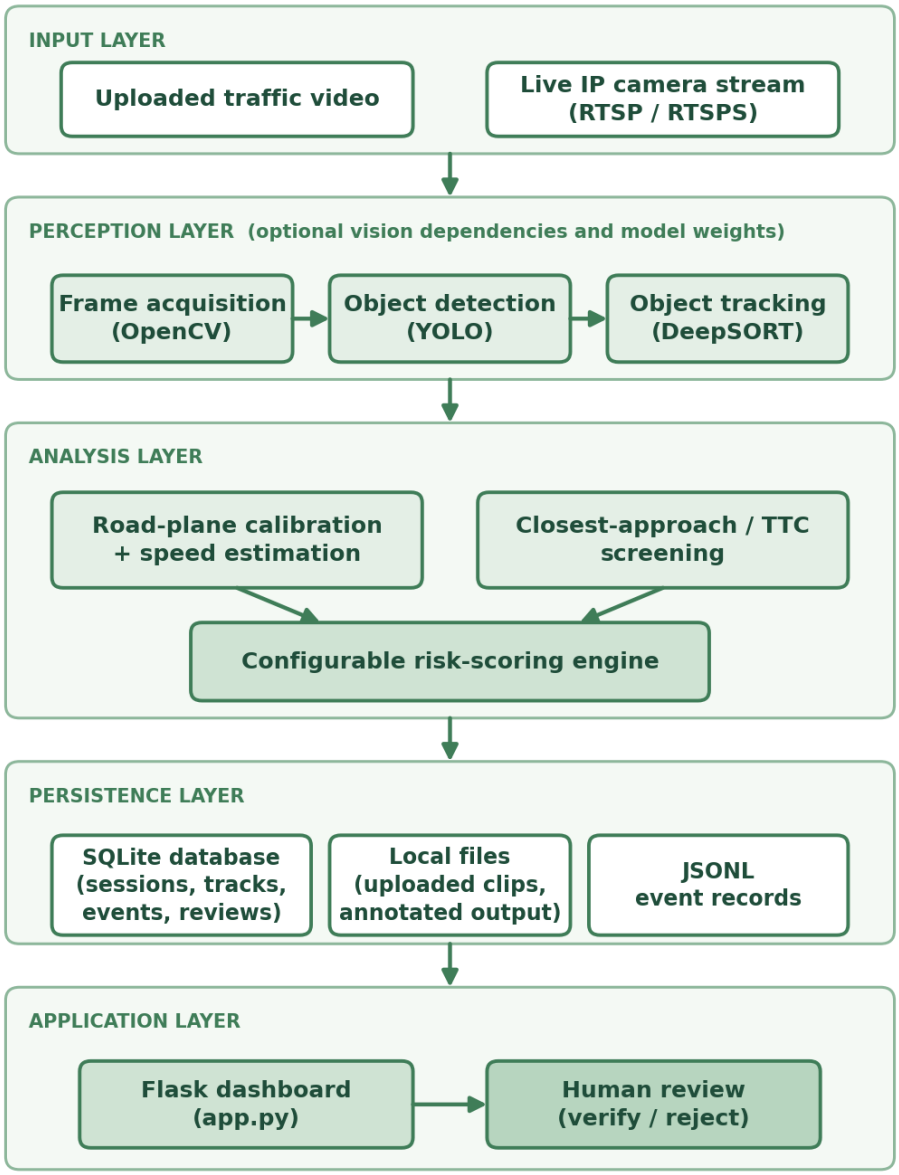


Figure 1. *Layered architecture of SAFE-Drive, from video input to human review.*

6.2 Architectural Flow

Figure 2 shows the ten-stage flow that a video follows through the system. Stages are shaded by the layer they belong to, matching Figure 1.

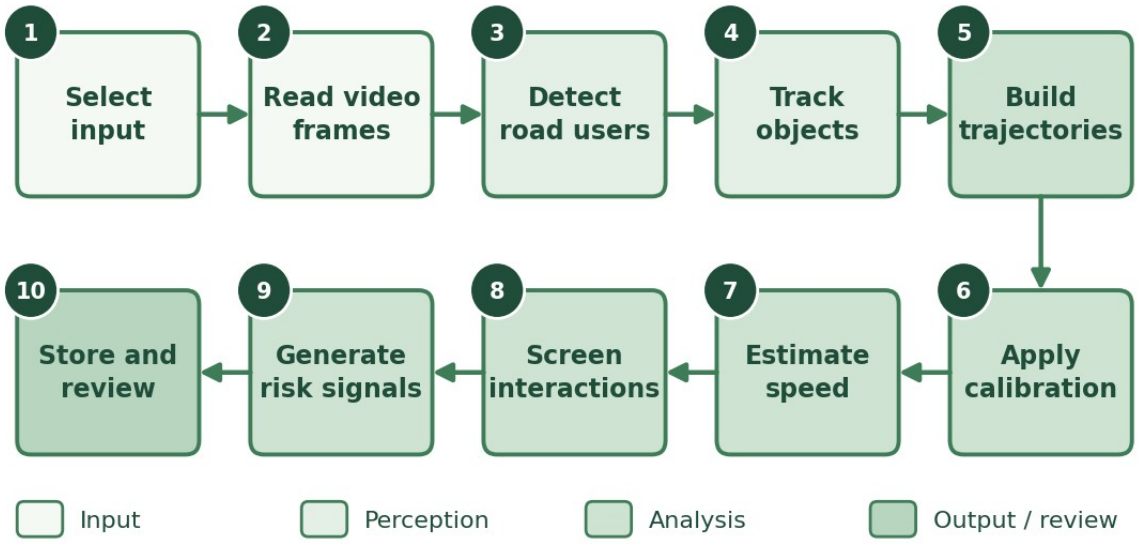


Figure 2. *End-to-end processing flow of SAFE-Drive (steps 1 to 10).*

Table 2. *Description of each stage in the processing flow*

Step	Stage	What happens
1	**Select input**	The user uploads a traffic video or supplies a supported live camera stream.
2	**Read frames**	The video-processing component reads footage frame by frame.
3	**Detect road users**	The YOLO-based detector identifies supported classes when compatible weights are available.
4	**Track objects**	DeepSORT links detections across frames and assigns session-local IDs.
5	**Build trajectories**	Tracked positions and timestamps are assembled into trajectories.
6	**Apply calibration**	With valid road-plane calibration, image coordinates map to real-world road coordinates.
7	**Estimate speed**	Calibrated trajectories give vehicle speed; without valid calibration, speed stays unavailable.
8	**Screen interactions**	Closest-approach and time-to-collision indicators are computed where trajectories allow.
9	**Generate risk signals**	Configurable rules combine the indicators into events that may need review.
10	**Store and review**	Sessions, tracks, events, and reviewer decisions are saved and shown in the dashboard.

7. Tools and Technologies

Tools used to build and run the project are listed separately from the libraries the application itself depends on.

7.1 Development Environment

Table 3. *Development tools*

Tool	Purpose
Python 3.10 or later	Main programming language for the application and analysis modules
Python virtual environment	Isolates project dependencies
`pyproject.toml`	Defines package metadata, dependencies, and the command-line entry point
macOS Terminal	Creating the environment, installing packages, and running the application
Git and GitHub	Version control and the public source-code repository [1]

7.2 Application Dependencies

Table 4. *Libraries the application depends on*

Library	Role in SAFE-Drive	Installation
Flask [7]	Web application and dashboard back end	Core
NumPy [8]	Numerical work: coordinate transforms, trajectory analysis	Core
SQLite [9]	Local relational storage; needs no database server	Core (bundled with Python)
OpenCV [6]	Reading and processing video frames and camera streams	Optional vision
Ultralytics (YOLO) [3]	Object-detection integration; needs compatible weights	Optional vision
DeepSORT [4]	Multi-object tracking and session-local track IDs	Optional vision

The optional vision dependencies are installed separately from the core dashboard dependencies, and compatible model files must be supplied for the matching features to run.

7.3 Interface and Data Formats

The dashboard is built with HTML, CSS, and JavaScript. Configuration and calibration data are stored as JSON, and structured event records use the line-oriented JSONL format.

8. AI Models and Algorithms

SAFE-Drive is a software system that integrates trained models with geometric and numerical algorithms. The project does not claim to have trained its YOLO detector or helmet model from scratch.

8.1 Component Descriptions

A. YOLO-Based Object Detection

Purpose: detect supported road users (cars, motorcycles, buses, trucks, pedestrians) in each frame. YOLO (You Only Look Once) is a family of object-detection models that predict locations and class labels from images [2]. It outputs bounding boxes and class predictions for the tracker. Real inference needs compatible pretrained weights, which the repository does not bundle.

> **Important:** a general-purpose pretrained YOLO model does not detect helmet violations. Helmet-state inference needs a separately trained, compatible model.

B. DeepSORT Multi-Object Tracking

Purpose: keep each road user's identity across frames. DeepSORT is a tracking-by-detection algorithm that associates detections using motion and appearance information [4]. A vehicle given ID 12 keeps that ID for the whole tracking session. It supplies trajectories to later modules but does not itself decide whether a vehicle is speeding or a collision has occurred.

C. Road-Plane Homography

Purpose: convert image coordinates into measured road-plane coordinates. A homography maps points between two planes [5]. SAFE-Drive builds a camera-specific transformation from at least four well-distributed, non-collinear point correspondences. It is a geometric technique, not a trained model.

D. Speed Estimation

Purpose: estimate vehicle speed from calibrated positions and timestamps. Accuracy depends on trajectory quality, timestamps, camera geometry, and calibration. Without valid calibration, detection and tracking continue but speed estimates are unavailable. This is a trajectory-based calculation, not a neural network.

E. Closest-Approach and Time-to-Collision Screening

Purpose: flag potentially hazardous interactions. Tracked positions and velocities yield closest-approach and time-to-collision indicators. These are screening aids for review, not proof that a collision occurred or will occur, and no trained accident-detection model is involved.

F. Configurable Risk-Scoring Engine

Purpose: combine indicators into interpretable risk signals that help prioritize events for human review. The weights are configurable research parameters, not trained accident probabilities or validated legal thresholds.

G. Optional Helmet-State Model

Purpose: experimental analysis of helmet-related visual information for tracked riders. It is enabled through the `SAFE_DRIVE_HELMET_MODEL` environment variable and needs compatible weights that follow the class order documented in `docs/HELMET_MODEL.md`. These weights and their dataset are not included, and the feature should not be described as a validated helmet-violation detector.

8.2 Models Versus Algorithms

Table 5. *Trained models compared with mathematical algorithms*

Component	Type	Needs trained weights?
YOLO detector	Deep-learning object-detection model	Yes
DeepSORT	Tracking algorithm; appearance features are implementation-dependent	Depends on the tracker's dependencies and configuration
Homography	Geometric transformation	No
Speed estimation	Numerical trajectory calculation	No
Closest-approach / TTC	Mathematical screening algorithm	No
Risk-scoring engine	Configurable rule-based calculation	No
Helmet-state inference	Optional trained classification/detection model	Yes

9. Backend Implementation

The implementation is organized into a web application, reusable analysis modules, configuration files, templates, static assets, and tests.

9.1 Application Layer

The back end is written in Python with Flask, which sits between the dashboard, the analysis workflow, and storage. The entry point is `app.py`, where `create_app()` builds the Flask application; reusable analysis code lives under `src/safe_drive/`. The back end coordinates analysis jobs, handles configuration, manages sessions, stores events, and returns results for review.

9.2 Video Processing Pipeline

When a job starts, the source and configuration go to the processing workflow, which reads frames from an uploaded video or an RTSP/RTSPS stream. If the optional vision dependencies and weights are present, frames go to detection and tracking. The resulting tracks provide positions and timestamps for calibrated speed estimation and trajectory-based risk screening. The workflow can produce annotated video, track summaries, and screening events.

9.3 Persistence

Sessions, vehicle and pedestrian tracks, events, and review decisions are stored in SQLite at `local_data/safe_drive.sqlite3`. SQLite suits a local research prototype because it needs no database server. Transient job status is kept in memory, and uploaded clips and generated outputs remain as files under `local_data/`.

9.4 Configuration

Camera settings come from a JSON file or from the environment variables `SAFE_DRIVE_CAMERA_URL`, `SAFE_DRIVE_CAMERA_ID`, and `SAFE_DRIVE_CAMERA_MODE`; helmet inference uses `SAFE_DRIVE_HELMET_MODEL`. The application does not load `.env` files automatically, so variables must be

set in the runtime environment. Camera credentials and private footage must not be committed to the public repository.

9.5 Event Logging and Human Review

Screening events are recorded and can be reviewed in the dashboard. A reviewer must enter a name or initials to record a decision, and events can be marked verified or rejected. These labels are review decisions only; they do not impose fines or establish legal responsibility.

10. Testing and Validation

Software correctness and the reliability of road safety measurements are two separate questions, and testing has to address both.

10.1 Automated Software Tests

The repository includes automated tests for camera configuration, geometric transformations, speed-estimation logic, temporal speed rules, track-registry behaviour, and SQLite session and event persistence. The local suite of 31 tests passed.

10.2 Evaluation Plan

Table 6. *Metrics for a complete evaluation*

Area	Recommended metrics
Detection	Precision, recall, and mAP at a declared IoU threshold
Tracking	HOTA or IDF1 for identity and trajectory consistency
Speed estimation	MAE and RMSE against independent speed measurements
Near-miss screening	Precision and recall against manually annotated ground truth with a predefined event definition
Calibration	Error against independent road distances or surveyed points
Performance	Latency and frames per second on documented hardware
Robustness	Different cameras, lighting, weather, traffic densities, and road geometries

Evaluation data should be split by camera or site and by time period to reduce leakage between training, validation, and testing. Numerical results should be reported only after the experiments have been run.

11. Results and Discussion

11.1 Software Test Results

Table 7. *Automated test coverage (31 tests, all passed)*

Area tested	Outcome
Camera configuration	Passed
Geometric transformations	Passed
Speed-estimation logic	Passed
Temporal speed rules	Passed
Track-registry behaviour	Passed
SQLite session and event persistence	Passed
Total	**31 of 31 tests passed**

> **Scope of these results:** these are software tests. They show that the tested components behave as designed, not that the system detects vehicles, tracks them, or estimates speed accurately on real footage.

11.2 End-to-End Video Test

Compatible detection weights and representative footage were not available at the time of testing, so no end-to-end figures are reported. Table 8 is the record sheet for a first small run on a real clip. Every value must come from actually running the pipeline, and entries stay blank until then.

Table 8. *End-to-end test record (to be completed after running a real test video)*

Item	Measurement	Result
Test video	Source, length, resolution, frame rate	—
Model weights	Detector file and version used	—
Calibration	Reference points used and calibration error	—
Detection	Road users counted by the system vs. counted by hand	—
Tracking	Number of tracks; identity switches seen on review	—
Speed	Estimated vs. reference speed for chosen vehicles	—
Near-miss screening	Events flagged; events confirmed or rejected by reviewer	—
Processing speed	Frames per second on the test hardware	—

11.3 Discussion

SAFE-Drive provides an integrated framework for traffic-video analysis, trajectory processing, configurable risk screening, and event review, and its modular design allows geometry, speed logic, track management, and database operations to be tested separately. Until the end-to-end run above is completed, no detection

accuracy, tracking accuracy, speed accuracy, near-miss detection rate, or safety benefit is claimed. Helmet-state inference is experimental, and accident detection is not implemented.

12. Limitations

1. End-to-end vision performance has not been validated on representative, labelled footage.
2. Real-world speed estimation requires camera-specific calibration.
3. Helmet-state inference is experimental and needs compatible trained weights; accident detection is not implemented.
4. Near-miss indicators and risk scores are screening aids, not proof of an accident, and the scoring weights are not validated probabilities or legal rules.
5. The application is for local research use and is not hardened for public network exposure; production use would need authentication, secure uploads, resource limits, camera-stream protections, privacy controls, and persistent storage for the database and uploaded files.
6. Live-camera performance depends on stream availability, codecs, network, and hardware, and automated results may include false positives and negatives that need human review.

13. Future Scope

1. Obtain licensed, annotated traffic-video datasets and validate detection and tracking with standard metrics.
2. Compare speed estimates with independently measured ground truth.
3. Define and validate a reproducible near-miss annotation protocol.
4. Train and validate a dedicated helmet-state classification model.
5. Evaluate across daylight, night, rain, traffic density, and camera viewpoints.
6. Add uncertainty estimates and per-indicator contribution reporting to the risk engine.
7. Add authentication, upload limits, retention controls, and safer deployment configuration, and optimize latency for live streams.
8. Run ethically reviewed studies before claiming any change in road-user behaviour or road safety outcomes.

14. Conclusion

SAFE-Drive is a modular research prototype for AI-assisted road safety monitoring. It brings together a Flask dashboard, optional YOLO detection and DeepSORT tracking, OpenCV video processing, road-plane calibration, timestamp-based speed estimation, trajectory-based near-miss screening, configurable risk scoring, and SQLite-backed event review. Its main contribution is integrating these parts with transparent risk indicators and human oversight instead of automatic enforcement.

It is not a single trained AI model: it is a software system that combines trained-model integration with tracking, geometry, numerical analysis, a database, and a web interface. Its 31 passing tests cover selected software components, but they do not replace model training records or end-to-end evaluation, which remain open work.

With representative datasets, compatible trained models, independent ground-truth measurements, and appropriate security controls, SAFE-Drive can serve as a foundation for further research into calibrated traffic analysis and uncertainty-aware road safety monitoring.

References

[1] SAFE-Drive, “SAFE-Drive: A research toolkit for AI-assisted road safety monitoring,” GitHub repository. [Online]. Available: https://github.com/bhuvzzs24-spec/SAFE-Drive

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788.

[3] Ultralytics, “YOLO documentation.” [Online]. Available: https://docs.ultralytics.com/

[4] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in Proc. IEEE Int. Conf. Image Processing (ICIP), 2017, pp. 3645–3649.

[5] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[6] OpenCV, “Open source computer vision library documentation.” [Online]. Available: https://docs.opencv.org/

[7] Flask, “Flask web framework documentation.” [Online]. Available: https://flask.palletsprojects.com/

[8] NumPy, “NumPy documentation.” [Online]. Available: https://numpy.org/doc/

[9] SQLite, “SQLite documentation.” [Online]. Available: https://www.sqlite.org/docs.html